NaLaPro
Beyond Words, Into Meaning

Indonesia AI
AI for Everyone, AI for Indonesia

Analisis Sentimen Pilpres 2019

By NalaPro

Indonesia.ai

Team NalaPro

01

Yasmin Kamila

02

Eko Erwis Wandoko

03

M Dhimas Agung Sugiharto

04

Muh. Idris

05

Wahid Setio Darmadi

Indonesia.ai

Objective (Tujuan Analisis)

Tujuan Utama

Menganalisis sentimen publik terhadap kedua pasangan calon presiden (Jokowi-Ma'ruf vs Prabowo-Sandi) pada masa Pemilu 2019 berdasarkan data Twitter.

Tujuan Khusus

1. Memahami karakteristik data tweet yang akan digunakan untuk pemodelan.
2. Membangun model klasifikasi sentimen (Positif, Negatif, Netral) menggunakan LSTM dan Random Forest.
3. Membandingkan performa kedua model untuk menentukan mana yang lebih efektif.

Data Understanding

Sumber Data

File: tweet.csv
Jumlah Data: 1.814 baris (tweet)
Periode Waktu: 13 - 15 April 2019 (Masa Tenang)

Struktur Data

Kolom	Tipe	Deskripsi
sentimen	String	Target: positif, negatif, netral
tweet	String	Fitur: Isi tweet

Kondisi Data

Tidak ada data hilang (missing values) pada kedua kolom.
Tidak ada data duplikat.

EDA - Distribusi & Panjang Teks

Distribusi Sentimen

01

Sentimen	Jumlah	Persentase
Netral	651	35.9%
Positif	609	33.6%
Negatif	554	30.5%

Data cukup **seimbang**.
Model tidak akan bias secara alami.

Analisis Panjang Teks

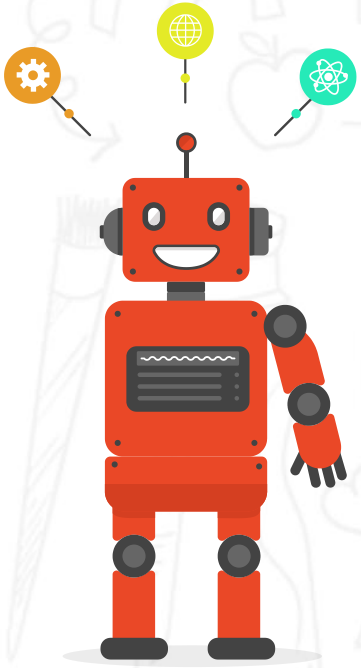
02

Rata-rata: **170-180 karakter**
Netral: cenderung lebih panjang (kutipan berita)
Positif/Negatif: lebih pendek (seruan/kritik)

Insight untuk Pemodelan:
- **LSTM:** Perlu *padding* untuk menyamakan panjang sequence.
- **Random Forest:** Tidak sensitif terhadap panjang teks setelah TF-IDF.

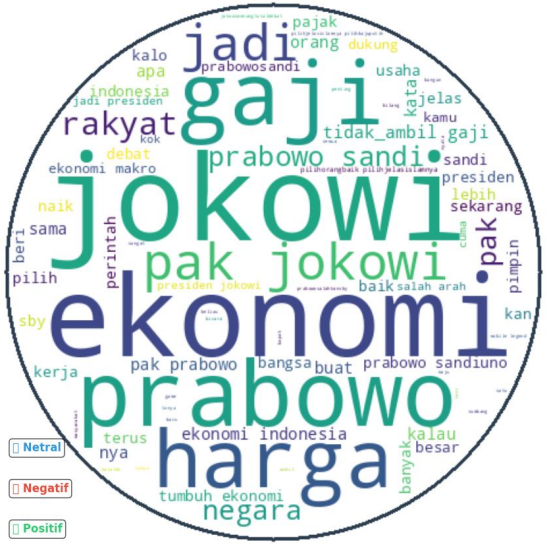
EDA - Wordcloud (Topik Utama)

Indonesia AI
AI for Everyone, AI for Indonesia



VISUALISASI WORDCLOUD TRAIN vs TEST

DATA TRAIN



DATA TEST

Keterangan:

✓ Dataset KONSISTEN & VALID
✓ Topik Utama: EKONOMI, PAJAK, TOKOH POLITIK
✓ Train: umum | Test: spesifik "tidak ambil gaji"
READY UNTUK ANALISIS SENTIMEN

Indonesia.ai

EDA - Mention & Hashtag

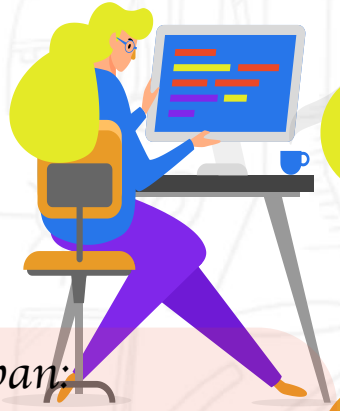
@ Analisis Mention (@username)

- **Terbanyak:** @jokowi, @prabowo, @sandiuno
- **Media Massa:** @kompascom, @detikcom, @cnnindonesia
- **Tokoh Publik:** @SBYudhoyono, @rockygerung

Analisis Hashtag (#)

- **Pro Jokowi** : #PilihOrangBaik, #PilihJelasIslamnya, #JokowiMenangTotalDebat
- **Pro Prabowo** : #UASdifitnahKejiBalasDiTPS, #TusukPrabowoSandi, #2019GantiPresiden
- **Netral** : #DebatPilpres2019, #TerimaKasihIstriku

Text Preprocessing Pipeline



01 Case Folding
UPPERCASE → lowercase

02 Data Cleaning
hapus @mention, #tag, link, angka, tanda baca

03 Normalisasi (Slang → Baku)
yg → yang, tdk → tidak

04 Stopword Removal
kecuali kata negasi (tidak, belum)

05 Stemming
menghargai → harga

06 Negation Handling
tidak baik → tidak_baik

Contoh Penerapan:

- **Original:** "@jokowi gak ngerti ekonomi makro!"
- **After Cleaning & Normalisasi:** "jokowi tidak mengerti ekonomi makro"Predict based on previous patterns
- **After Stemming & Negation:** "jokowi tidak_ngerti ekonomi makro"

Indonesia.ai

WordNinja

(di terapkan untuk Random Forest)

Apa itu WordNinja?

WordNinja adalah library Python yang digunakan untuk memisahkan kata yang tergabung (concatenated words) menjadi kata-kata terpisah.

Contoh:" "pilihorangbaik"→ "pilih orang baik"

Ini sangat berguna untuk memproses hashtag yang sering muncul di tweet.

Penerapan pada Data Tweet

Original: "Ayo coblos #PilihOrangBaik #JokowiLagi"
Setelah WordNinja: "Ayo coblos pilih orang baik jokowi lagi"

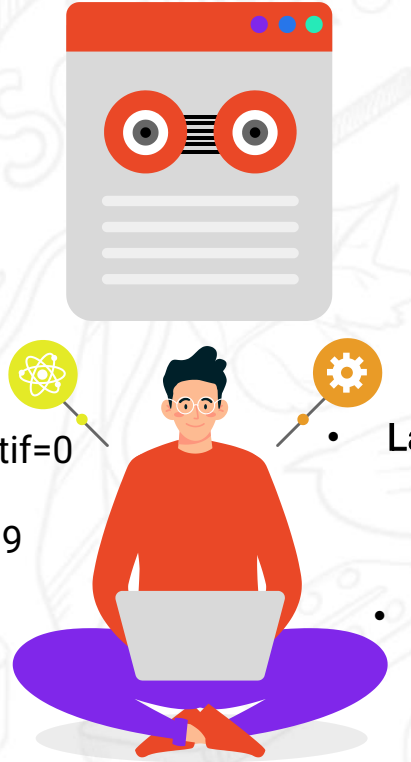
Dampak terhadap Model Random Forest

Fitur Lebih kaya
#PilihOrangBaik → "pilih orang baik" ➜ 1 kata ➜ 3 kata bermakna ➜ Informasi bertambah di matriks TF-IDF
Representasi Teks Meningkat
Kata kunci kampanye muncul terpisah: "pilih", "orang", "baik" ➜ Model lebih paham konteks
Krusial untuk Random Forest
Random Forest sangat bergantung pada TF-IDF
➜ Pemisahan kata = fitur lebih banyak & akurat
note: LSTM dapat mempelajari pola langsung dari teks mentah
➜ Tidak terlalu membutuhkan WordNinja

Feature Engineering & Modeling

A. Random Forest

- **Label Encoding:** positif=2, netral=1, negatif=0
- **TF-IDF Vectorizer:** max_features=3000, ngram_range=(1,2), min_df=3, max_df=0.9
- **Model:** RandomForestClassifier dengan GridSearchCV



B. LSTM

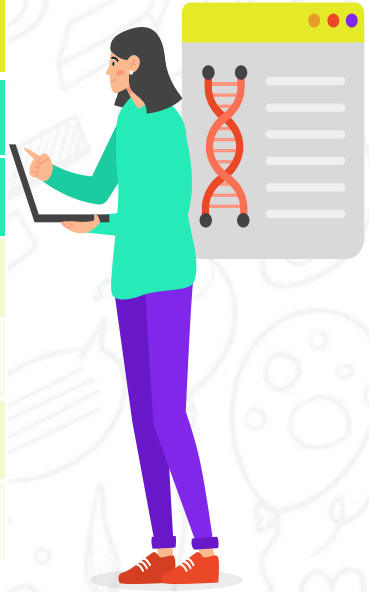
- **Label Encoding:** sama seperti Random Forest
- **Tokenisasi & Padding:** max_len=100
- **Arsitektur:** Embedding → LSTM(128) → Dropout(0.5) → Dense(64) → Dense(3)
- **Training:** Early Stopping, AdamW optimizer

Logbook Diagnostik Random Forest (V1 - V6)

V1/V2: Baseline	V3: Pembersihan	V4: TF-IDF Limit	V5: Deep Tree	V6: Validasi
`GridSearch (f1_weighted)`	`Penambahan WordNinja`	`max_features=2000`	`n_estimators=500, max_depth=20`	`Cross Validation (CV=10)`
Status: Performa Puncak (0.57)	**Status:** Akurasi stagnan, walau teks rapi	**Status:** Akurasi menurun. Ditolak	**Status:** Overfitting. Akurasi turun ~3%	**Status:** Akurasi turun 1%

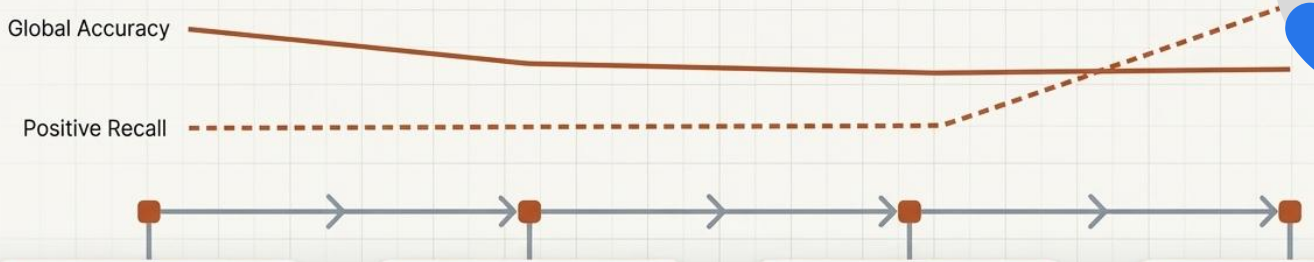
Hasil Perbandingan Model Random Forest

Model	TD-IDF Features	Hyperparameter	Akurasi	F1-Score
RF_V1	3000, (1,2)-gram	GridSearch (F1)	57.3 %	0.57
RF_V2	3000, (1,2)-gram	GridSearch Multi-Score	57.3 %	0.57
RF_V3	2000, (1,3)-gram	GridSearch (F1)	56.7 %	0.56
RF_V4	3000, (1,2)-gram	Manual (n=500, dll)	54.0 %	0.54
RF_V5	3000, (1,2)-gram	Manual (n=100, dll)	55.6 %	0.55
RF_V6	3000, (1,2)-gram	GridSearch (F1, cv=10)	56.5 %	0.56



Kesimpulan Random Forest: Model terbaik **RF_V1 / RF_V2 (57.3%). GridSearch** berhasil menemukan parameter optimal. ngram=(1,3) justru sedikit menurunkan performa.

Logbook Diagnostik LSTM (V1 - V4)



V1: Baseline Model

15 Epochs, AdamW 0.001

Status: Akurasi awal terbentuk

V2: Early Stopping

monitor='val_loss', patience=3

Status: Akurasi 56%. Menghentikan overfitting dini.

V3: Kompresi Sekuens

Padding max_len dipotong 100 -> 50

Status: Akurasi stagnan, tapi Recall Positif mulai naik (+0.03).

V4: GridSearch Tuning

Tuning optimal pada units, dropout, lr

Status: Val Accuracy 58%. Lonjakan drastis pada metrik Recall.

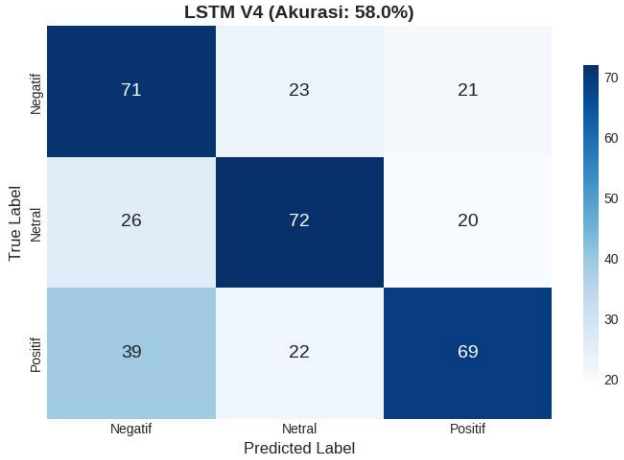
Hasil Perbandingan Model lSTM

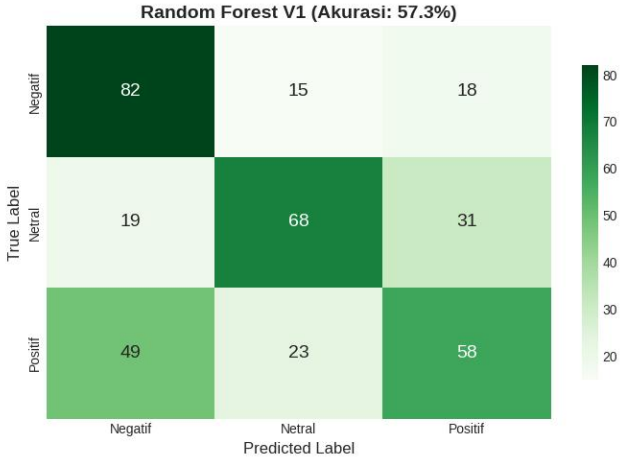
Model	Arsitektur / Learning Rate	Akurasi	F1-Score
LSTM_V1	LSTM-128, Dropout(0.5), lr=0.001	58.9%	0.58
LSTM_V2	LSTM-128, Dropout(0.5), lr=0.001, ES	56.0%	0.54
LSTM_V3	LSTM-128, Dropout(0.5), lr=0.001	54.0%	0.54
LSTM_V4	LSTM-32, Dropout(0.4), lr=0.0005, ES	58.0%	0.58

Kesimpulan LSTM:
Model terbaik adalah **LSTM_V4 (58%).** Penurunan learning rate dan penyesuaian dropout meningkatkan stabilitas. Early Stopping efektif cegah overfitting.

Perbandingan Model Terbaik: LSTM vs Random Forest

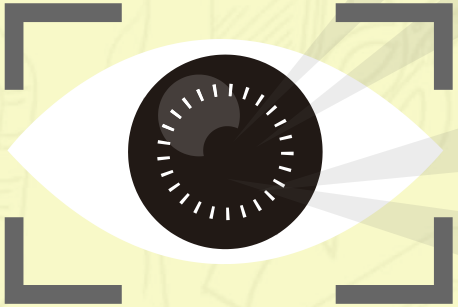
Aspek	LSTM (V4)	Random Forest (V1)
Akurasi	58.0%	57.3%
F1-Score (Weighted)	0.58	0.57
Precision (Macro)	0.58	0.56
Recall (Macro)	0.58	0.56





Analisis Hasil & Pembahasan

Kesimpulan Utama



LSTM Unggul Tipis

Selisih akurasi 0.7% (58.0% vs 57.3%). LSTM sedikit lebih baik dalam menangkap konteks urutan kata.

Kinerja Hampir Setara

Data mungkin tidak memiliki pola sekuensial yang terlalu kompleks, sehingga Random Forest dengan TF-IDF sudah bekerja baik.

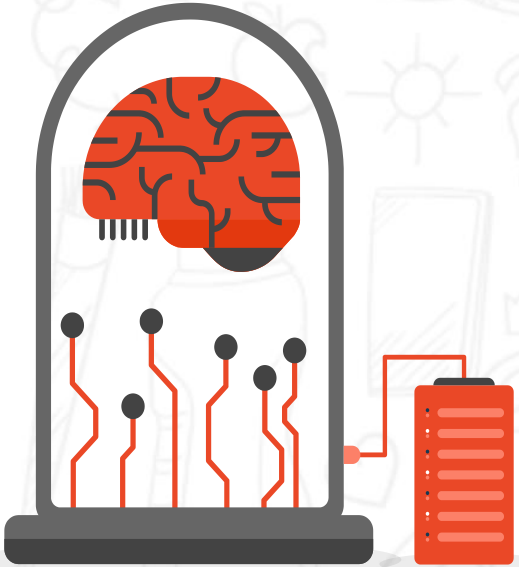
Kesulitan Memprediksi Sentimen Positif

Kedua model paling sering salah mengklasifikasikan sentimen Positif (sarkasme/metafora).

Kekuatan Model

Random Forest unggul di prediksi Negatif (recall tinggi), LSTM lebih seimbang antar kelas.

Kesimpulan Akhir



01 Data Bersih & Seimbang

Dataset dalam kondisi baik dan ideal untuk pemodelan.

02 Isu Ekonomi Dominan

Harga, gaji menjadi topik sentral percakapan.

03 Preprocessing Krusial

Normalisasi slang dan negation handling sangat penting untuk bahasa informal.

04 LSTM sebagai Juara

Model LSTM_V4 menjadi yang terbaik dengan akurasi 58.0%, mengungguli Random Forest (57.3%).

05 Ruang untuk Improvement

Akurasi ~58% menunjukkan tantangan analisis sentimen media sosial. Peningkatan: tambah fitur eksternal, gunakan model transformer (IndoBERT), atau lakukan ensemble kedua model.

Thank you

Indonesia.ai

Indonesia.ai